

[Página Principal](#) / [Mis cursos](#) / [TUPProgl](#) / [COMISIÓN NOCHE-PARCIALES-2026](#) / [Cuestionario 2° Parcial-Comisión Noche- 2026](#)

Comenzado el Friday, 29 de May de 2026, 18:27

Estado Finalizado

Finalizado en Friday, 29 de May de 2026, 20:10

Tiempo empleado 1 hora 43 minutos

Calificación Sin calificar aún

Pregunta **1**

Correcta

Se puntúa 2,00 sobre 2,00

Establecer la relación entre el concepto y su definición:

Conjunto finito ordenado de elementos homogéneos

Arreglo



Secuencia de caracteres.

Cadena



Es una estructura de datos
que agrupa variables que pueden tener tipos
diferentes

Estructura de Registros



Respuesta correcta

La respuesta correcta es: Conjunto finito ordenado de elementos homogéneos → Arreglo, Secuencia de caracteres. → Cadena, Es una estructura de datos que agrupa variables que pueden tener tipos diferentes → Estructura de Registros

Pregunta **2**

Correcta

Se puntúa 2,00 sobre 2,00

Un arreglos es  porque hay un número específico de elementos en el arreglo,

Respuesta correcta

La respuesta correcta es:

Un arreglos es [finito] porque hay un número específico de elementos en el arreglo,

Pregunta 3

Parcialmente correcta

Se puntúa 0,67 sobre 2,00

Las ESTRUCTURAS DE DATOS  son aquellas en las que el tamaño ocupado en la memoria se define  de que el programa se ejecute y  puede ser modificado durante la ejecución del programa.

Respuesta parcialmente correcta.

Ha seleccionado correctamente 1.

La respuesta correcta es:

Las ESTRUCTURAS DE DATOS [ESTÁTICAS] son aquellas en las que el tamaño ocupado en la memoria se define [antes] de que el programa se ejecute y [no] puede ser modificado durante la ejecución del programa.

Pregunta 4

Parcialmente correcta

Se puntúa 0,67 sobre 2,00

Indicar las estructuras que corresponden a Estructuras estáticas:

- ☒ a. Pila
- ☐ b. Árboles
- ☐ c. Boolean
- ☒ d. Cadena
- ☐ e. Archivos
- ☐ f. Grafos



Respuesta parcialmente correcta.

Ha seleccionado correctamente 1.

Las respuestas correctas son: Boolean, Cadena, Archivos

Pregunta 5

Parcialmente correcta

Se puntúa 0,67 sobre 2,00

Indicar las estructuras que corresponden a Estructuras Dinámicas:

- ☐ a. Archivos
- ☒ b. Boolean
- ☐ c. Cadena
- ☐ d. Pila
- ☐ e. Grafos
- ☒ f. Árboles

✗

✓

Respuesta parcialmente correcta.

Ha seleccionado correctamente 1.

Las respuestas correctas son: Pila, Grafos, Árboles

Pregunta 6

Correcta

Se puntúa 2,00 sobre 2,00

Indicar cual o cuales de las siguientes frases corresponden a características de los arreglos:

- ☐ a. Ninguna Opción
- ☐ b. Un arreglo no es finito.
- ☐ c. Un arreglo solo puede ser de una dimensión
- ☒ d. •Para acceder a un elemento en particular de un arreglo se usa el nombre del arreglo seguido de un índice entre corchetes.
- ☒ e. •Un arreglo puede tener una o varias dimensiones .

✓

✓

Respuesta correcta

Las respuestas correctas son:

•Un arreglo puede tener una o varias dimensiones .

•Para acceder a un elemento en particular de un arreglo se usa el nombre del arreglo seguido de un índice entre corchetes.

Pregunta 7

Correcta

Se puntúa 2,00 sobre 2,00

Indicar cual o cuales de las siguientes frases respecto a los arreglos son falsas:

- ☒ a. Un arreglo no es finito. ✓
- ☒ b. Ninguna Opción ✗
- ☐ c. •Un arreglo puede tener una o varias dimensiones .
- ☒ d. Un arreglo solo puede ser de una dimensión ✓
- ☐ e. •Para acceder a un elemento en particular de un arreglo se usa el nombre del arreglo seguido de un índice entre corchetes.

Respuesta correcta

Las respuestas correctas son: Un arreglo solo puede ser de una dimensión, Un arreglo no es finito.

Pregunta 8

Correcta

Se puntúa 2,00 sobre 2,00

Una Estructura de datos en una colección de datos que se caracterizan por su organización y las operaciones que se definen en ella.

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta 9

Correcta

Se puntúa 2,00 sobre 2,00

Indicar si el siguiente trozo de Algoritmo corresponde a la carga de elementos en un arreglo:

para (i = 0; i < 10; i:=i+1)

 escribir("Ingresar el elemento");

 leer(a);

 lista[i]:=a;

fin para

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta **10**

Correcta

Se puntúa 2,00 sobre 2,00

La búsqueda binaria se puede aplicar solamente cuando el arreglo está ordenado.

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta **11**

Correcta

Se puntúa 2,00 sobre 2,00

En la búsqueda binaria, se toma un elemento central, normalmente el elemento que se encuentra a la mitad del arreglo, y se lo compara con el elemento buscado. Si el elemento buscado es menor, se toma el intervalo que va desde el elemento central al principio, en caso contrario, se toma el intervalo que va desde el elemento central hasta el final del intervalo.

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta 12

Correcta

Se puntúa 2,00 sobre 2,00

¿Cuáles de estas características son válidas en el algoritmo de Búsqueda Binaria?

- ☒ a. Este algoritmo reduce el tiempo de búsqueda considerablemente, ya que disminuye exponencialmente el número de iteraciones necesarias. ✓
- ☒ b. Se utiliza cuando la estructura de datos en el que queremos determinar la existencia de un elemento esté previamente ordenado. ✓
- ☒ c. Para implementar este algoritmo se compara el elemento a buscar con un elemento central del vector, si este es mayor que el del elemento buscado se repite el procedimiento en la parte del vector que va desde el inicio de éste hasta el elemento tomado, en caso contrario se toma la parte del vector que va desde el elemento tomado hasta el final. ✓
- ☐ d. Para implementar este algoritmo se compara el elemento a buscar con un elemento inicial del vector, si este es mayor que el del elemento buscado se repite el procedimiento en la parte del vector que va desde el inicio de éste hasta el elemento tomado, en caso contrario se toma la parte del vector que va desde el elemento tomado hasta el final.
- ☐ e. No se requiere que la estructura de datos en el que queremos determinar la existencia de un elemento esté previamente ordenado.

Respuesta correcta

Las respuestas correctas son: Se utiliza cuando la estructura de datos en el que queremos determinar la existencia de un elemento esté previamente ordenado., Este algoritmo reduce el tiempo de búsqueda considerablemente, ya que disminuye exponencialmente el número de iteraciones necesarias. , Para implementar este algoritmo se compara el elemento a buscar con un elemento central del vector, si este es mayor que el del elemento buscado se repite el procedimiento en la parte del vector que va desde el inicio de éste hasta el elemento tomado, en caso contrario se toma la parte del vector que va desde el elemento tomado hasta el final.

Pregunta 13

Parcialmente correcta

Se puntúa 1,00 sobre 2,00

¿Cuáles de estas características son válidas en el algoritmo de Búsqueda Secuencial o Lineal?

- ☐ a. Es el método de búsqueda más rápido porque recorre los valores desde el comienzo de la estructura.
- ☐ b. Retorna el valor buscado dentro de la estructura.
- ☒ c. Utiliza un ciclo iterativo y se comienza a recorrer la estructura hasta encontrar el valor buscado. ✓
- ☐ d. Se recorre la estructura hasta el final y si se encuentra otro valor igual al buscado, se retornan todos los valores.
- ☐ e. Retorna el índice de la estructura en donde se encuentra el valor buscado.

Respuesta parcialmente correcta.

Ha seleccionado correctamente 1.

Las respuestas correctas son: Utiliza un ciclo iterativo y se comienza a recorrer la estructura hasta encontrar el valor buscado., Retorna el índice de la estructura en donde se encuentra el valor buscado.

Pregunta **14**

Parcialmente correcta

Se puntúa 1,33 sobre 2,00

¿Cuáles de estas características son válidas en el algoritmo de Ordenamiento [Burbuja](#)?

- ☐ a. Solo permite comparar valores numéricos enteros.
- ☐ b. Es uno de los métodos más simples.
- ☒ c. Consiste en comparar todos los elementos de una lista contra todos, si se cumple que uno es mayor o menor que otro, entonces se intercambia de posición. ✓
- ☐ d. Es un algoritmo que funciona con recursividad.
- ☒ e. Los valores dentro del arreglo no deben estar ordenados. ✓

Respuesta parcialmente correcta.

Ha seleccionado correctamente 2.

Las respuestas correctas son: Es uno de los métodos más simples., Consiste en comparar todos los elementos de una lista contra todos, si se cumple que uno es mayor o menor que otro, entonces se intercambia de posición., Los valores dentro del arreglo no deben estar ordenados.

Pregunta **15**

Parcialmente correcta

Se puntúa 1,33 sobre 2,00

¿Cuáles de las siguientes características no son válidas en [Matrices](#)?

- ☐ a. Solamente hay valores en las posiciones válidas.
- ☒ b. El tamaño puede variar en tiempo de ejecución. ✓
- ☐ c. Sus índices se deben expresar entre corchetes.
- ☐ d. Una referencia a una posición fuera del rango no produce un error.
- ☒ e. No requieren definir un tipo de datos en su declaración. ✓

Respuesta parcialmente correcta.

Ha seleccionado correctamente 2.

Las respuestas correctas son: No requieren definir un tipo de datos en su declaración., Una referencia a una posición fuera del rango no produce un error., El tamaño puede variar en tiempo de ejecución.

Pregunta **16**

Finalizado

Se puntúa como 0 sobre 12,00

En base al programa publicado en el repositorio

<https://github.com/facundouferer/cursodec/blob/main/Parciales/Parcial02/segundoParcialAjedrez.c> desarrolle la función Buscar Pieza por búsqueda Binaria.

La pieza debe buscarse utilizando su posición en el tablero, representada por el vector `posicion`, donde `posicion[0]` es la fila y `posicion[1]` es la columna.

Para que el programa funcione perfectamente debe existir ésta función y no alterar el código fuera de la función.

```
struct buscarPieza(struct Pieza piezas[], int cantidad,int fila, int columna) {
    int inicio = 0, fin = -1;
    while (inicio <= fin) {
        int medio = (inicio + fin) / 2;
        if (piezas[medio].posicion == )
            printf("Pieza encontrada posicion : %d%d\n",piezas[medio].posicion);
        break;
        return medio;
    else if (piezas[medio].posicion < fila && piezas[medio].posicion < columna)
        inicio = medio + 1;
    else
        fin = medio - 1;
    }
    return -1;
};
```

Pregunta **17**

Finalizado

Se puntúa como 0 sobre 9,00

En base al programa publicado en el repositorio

<https://github.com/facundouferer/cursodec/blob/main/Parciales/Parcial02/segundoParcialAjedrez.c> desarrolle la función Capturar Pieza.

La función debe buscar una pieza por su posición (`fila`, `columna`) y marcarla como capturada cambiando el campo `activa` a 0.

Para que el programa funcione perfectamente debe existir ésta función y no alterar el código fuera de la función.

```
void capturarPieza(struct Pieza piezas[],int cantidad,int fila,int columna) {
    for(int i = 0; i < cantidad; i++)
        if(piezas[i].posicion[0]== fila && piezas[i].posicion[1]== columna){
            piezas[i].activa=0;
            printf("Pieza que se encuentra en la posicion %d%d capturada"),fila,columna;
            return 0;
        }
    printf ("no se encontro la pieza");
}
```


Pregunta **18**

Finalizado

Se puntúa como 0 sobre 9,00

En base al programa publicado en el repositorio

<https://github.com/facundouferer/cursodec/blob/main/Parciales/Parcial02/segundoParcialAjedrez.c> desarrolle la función Listar Piezas Activas.

La función debe mostrar solamente las piezas que continúan activas en el tablero.

Para que el programa funcione perfectamente debe existir ésta función y no alterar el código fuera de la función.

```
void listarPiezasActivas(struct Pieza piezas[], int cantidad)
{
    printf("\nPiezas activas:\n");
    int encontrado = 0;
    for (int i = 0; i < cantidad; i++)
        if (piezas[i].activa == 1)
        {
            printf("sE encuentra en la posicion (%d,%d)",piezas[i].posicion[0], piezas[i].posicion[1];
            encontrado++;
        }
    if (encontrado == 0)
        printf("No hay piezas activas.\n");
}
```

Pregunta **19**

Finalizado

Se puntúa como 0 sobre 9,00

En base al programa publicado en el repositorio

<https://github.com/facundouferer/cursodec/blob/main/Parciales/Parcial02/segundoParcialAjedrez.c> desarrolle la función Listar Piezas Por Posición.

La función debe listar las piezas ordenadas primero por fila y luego por columna.

Para que el programa funcione perfectamente debe existir ésta función y no alterar el código fuera de la función.

```
void listarPiezasPorPosicion(struct Pieza piezas[], int cantidad)
{
    for (int i = 0; i < cantidad - 1; i++)
        for (int j = 0; j < cantidad - i - 1; j++)
            if (piezas[j].posicion[0] > piezas[j + 1].posicion[0] ||
                (piezas[j].posicion[0] == piezas[j + 1].posicion[0] &&
                 piezas[j].posicion[1] > piezas[j + 1].posicion[1]))
            {
                aux = piezas[j];
                piezas[j] = piezas[j + 1];
                piezas[j + 1] = aux;
            }

    printf("\nPiezas ordenadas:\n");
    for (int i = 0; i < cantidad; i++)
        printf("Posicion: %d,%d",piezas[i].posicion[0], piezas[i].posicion[1]);
}
```

Pregunta **20**

Finalizado

Se puntúa como 0 sobre 9,00

En base al programa publicado en el repositorio

<https://github.com/facundouferer/cursodec/blob/main/Parciales/Parcial02/segundoParcialAjedrez.c> desarrolle la función Listar Piezas por Valor.

La función debe ordenar las piezas de mayor a menor valor y luego mostrarlas por pantalla.

Para que el programa funcione perfectamente debe existir ésta función y no alterar el código fuera de la función.

```
void listarPiezasPorValor(struct Pieza piezas[], int cantidad)
{
    for (int i = 0; i < cantidad - 1; i++)
        for (int j = 0; j < cantidad - i - 1; j++)
            if (piezas[j].valor < piezas[j + 1].valor)
            {
                aux = piezas[j];
                piezas[j] = piezas[j + 1];
                piezas[j + 1] = aux;
            }
    printf("\nPiezas ordenadas por valor:\n");
    for (int i = 0; i < cantidad; i++)
        printf("Posicion: (%d,%d)", piezas[i].posicion[0], piezas[i].posicion[1]);
}
```

Pregunta **21**

Finalizado

Se puntúa como 0 sobre 12,00

En base al programa publicado en el repositorio

<https://github.com/facundouferer/cursodec/blob/main/Parciales/Parcial02/segundoParcialAjedrez.c> desarrolle la función Modificar Pieza, que permita modificar el tipo y el color.

El tipo de pieza no debe ingresarse manualmente como texto: debe seleccionarse con un número y luego obtenerse usando la función **seleccionarTipoDePieza**.

El valor tampoco debe ingresarse manualmente: debe calcularse automáticamente usando la función **obtenerValorDePieza**.

El color debe cargarse como un caracter: **b** para blanco y **n** para negro.

Para que el programa funcione perfectamente debe existir ésta función y no alterar el código fuera de la función.

```
void modificarPieza(struct Pieza piezas[], int cantidad, int fila, int columna)
{
    for (int i = 0; i < cantidad; i++)
        if (piezas[i].posicion[0] == fila && piezas[i].posicion[1] == columna)
        {
            printf("Asignar fila: ");
            scanf("%d", &piezas[i].posicion[0]);
            printf("Asignar columna: ");
            scanf("%d", &piezas[i].posicion[1]);
            printf("seleccione el color: ");
            printf("| 'b' blanco | 'n' negro |");
            scanf(" %c", &piezas[i].color);
            printf("Pieza modificada correctamente.\n");
            return;
        }
    else{
        printf("no se encontro");
    }
}
```

◀ Notas Parciales - C3-2026

Ir a...

Desafío Preparación 2° Parcial Com 3 ▶